

ADVANCED SQL QUERIES FOR BUSINESS REPORTS

1. JOIN Queries

Query 1: Customer Order Report

Display customer details along with their orders.

SELECT

C.Customer_ID,

C.Customer_Name,

O.Order_ID,

O.Order_Date,

O.Order_Status,

O.Total_Amount

FROM Customer C

INNER JOIN Orders O

ON C.Customer_ID = O.Customer_ID;

The screenshot shows a database management tool interface. On the left is a 'SCHEMAS' tree with a search bar and a list of databases including 'college', 'collegedb', 'company', 'ecommerce_order', and 'ecommerce_order_mana'. The 'ecommerce_order_mana' database is expanded, showing tables like 'category', 'customer', 'order_details', 'orders', 'payment', 'product', 'seller', and 'shipping'. The main window displays the SQL query for 'Query 1' in a text editor. Below the editor is a 'Result Grid' showing the results of the query. The grid has columns for Customer_ID, Customer_Name, Order_ID, Order_Date, Order_Status, and Total_Amount. The results show five rows of data for customers 101 through 105.

Customer_ID	Customer_Name	Order_ID	Order_Date	Order_Status	Total_Amount
101	Arun Kumar	501	2026-04-01	Delivered	799.00
102	Priya Sharma	502	2026-04-03	Shipped	2499.00
103	Rahul Raj	503	2026-04-05	Confirmed	1198.00
104	Divya S	504	2026-04-08	Pending	850.00
105	Karthik M	505	2026-04-10	Delivered	2449.00

Query 2: Product Category and Seller Report

Display products along with their category and seller information.

SELECT

P.Product_ID,

P.Product_Name,

C.Category_Name,

S.Seller_Name,

P.Price,

P.Stock_Quantity,

P.Product_Status

FROM Product P

INNER JOIN Category C

ON P.Category_ID = C.Category_ID

INNER JOIN Seller S

ON P.Seller_ID = S.Seller_ID;

The screenshot shows a database management tool interface. On the left, the 'SCHEMAS' panel displays a tree view of database objects, including tables like 'category', 'customer', 'order_details', 'orders', 'payment', 'product', 'seller', 'shipping', 'Views', 'Stored Procedures', 'Functions', 'jdbc_demo', and 'sakila'. The 'ecommerce_order_managem' schema is selected. The main query editor displays the following SQL query:

```
2 P.Product_ID,  
3 P.Product_Name,  
4 C.Category_Name,  
5 S.Seller_Name,  
6 P.Price,  
7 P.Stock_Quantity,  
8 P.Product_Status  
9 FROM Product P  
10 INNER JOIN Category C  
11 ON P.Category_ID = C.Category_ID  
12 INNER JOIN Seller S  
13 ON P.Seller_ID = S.Seller_ID;
```

Below the query editor, the 'Result Grid' shows the results of the query. The grid has columns: Product_ID, Product_Name, Category_Name, Seller_Name, Price, Stock_Quantity, and Product_Status. The results are as follows:

Product_ID	Product_Name	Category_Name	Seller_Name	Price	Stock_Quantity	Product_Status
401	Wireless Mouse	Electronics	Tech World	799.00	50	Available
402	Bluetooth Headphones	Electronics	Tech World	2499.00	30	Available
403	Cotton T-Shirt	Clothing	Fashion Hub	599.00	100	Available
404	Denim Jeans	Clothing	Fashion Hub	1499.00	40	Available
405	Database Management Book	Books	Book Store	850.00	25	Available
406	Java Programming Book	Books	Book Store	950.00	20	Available

Query 3: Complete Order Report Using Multiple JOINS

Generate a detailed business report containing customer, order, product, and payment information.

SELECT

C.Customer_Name,

O.Order_ID,

O.Order_Date,

P.Product_Name,

OD.Quantity,

OD.Subtotal,

PAY.Payment_Method,

PAY.Payment_Status

FROM Customer C

INNER JOIN Orders O

ON C.Customer_ID = O.Customer_ID

INNER JOIN Order_Details OD

ON O.Order_ID = OD.Order_ID

INNER JOIN Product P

ON OD.Product_ID = P.Product_ID

INNER JOIN Payment PAY

ON O.Order_ID = PAY.Order_ID;

The screenshot shows a database query editor interface. On the left is a 'Navigator' pane with a 'SCHEMAS' tree. The 'ecommerce_order_managem' schema is expanded, showing tables like 'category', 'customer', 'order_details', 'orders', 'payment', 'product', 'seller', 'shipping', 'Views', 'Stored Procedures', 'Functions', 'jdbc_demo', and 'sakila'. The 'Administration' tab is selected. The main area displays 'Query 1' with the following SQL code:

```
8      PAY.Payment_Method,  
9      PAY.Payment_Status  
10     FROM Customer C  
11     INNER JOIN Orders O  
12     ON C.Customer_ID = O.Customer_ID  
13     INNER JOIN Order_Details OD  
14     ON O.Order_ID = OD.Order_ID  
15     INNER JOIN Product P  
16     ON OD.Product_ID = P.Product_ID  
17     INNER JOIN Payment PAY  
18     ON O.Order_ID = PAY.Order_ID;  
19
```

Below the query is the 'Result Grid' showing 8 columns: Customer_Name, Order_ID, Order_Date, Product_Name, Quantity, Subtotal, Payment_Method, and Payment_Status. The results are as follows:

Customer_Name	Order_ID	Order_Date	Product_Name	Quantity	Subtotal	Payment_Method	Payment_Status
Arun Kumar	501	2026-04-01	Wireless Mouse	1	799.00	UPI	Successful
Priya Sharma	502	2026-04-03	Bluetooth Headphones	1	2499.00	Credit Card	Successful
Rahul Raj	503	2026-04-05	Cotton T-Shirt	2	1198.00	Debit Card	Successful
Divya S	504	2026-04-08	Database Management Book	1	850.00	UPI	Pending
Karthik M	505	2026-04-10	Denim Jeans	1	1499.00	Net Banking	Successful
Karthik M	505	2026-04-10	Java Programming Book	1	950.00	Net Banking	Successful

2. Set Operations

Query 4: UNION

Display all unique email addresses from customers and sellers.

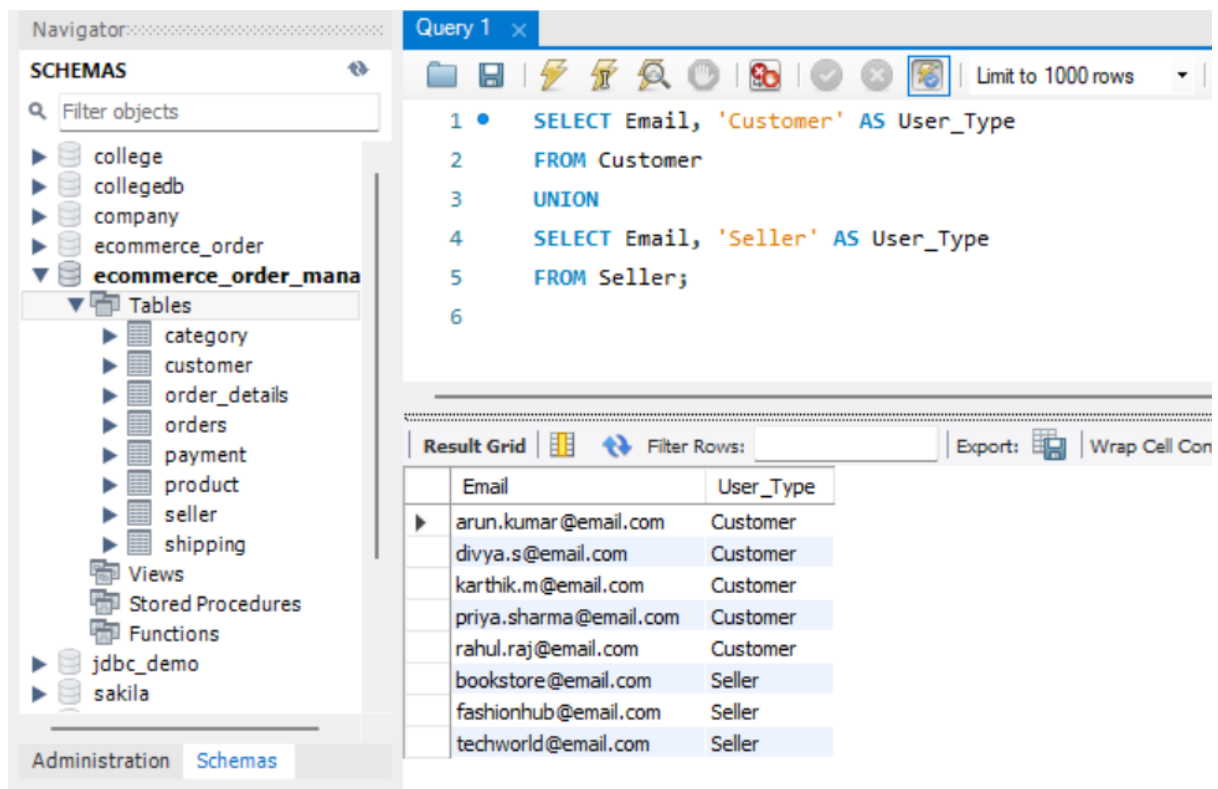
```
SELECT Email, 'Customer' AS User_Type
```

```
FROM Customer
```

```
UNION
```

```
SELECT Email, 'Seller' AS User_Type
```

```
FROM Seller;
```



The screenshot shows a database IDE interface. On the left is a 'Navigator' pane with a tree view of database objects. The 'ecommerce_order_mana' schema is expanded, showing tables like 'category', 'customer', 'order_details', 'orders', 'payment', 'product', 'seller', and 'shipping'. The main area is titled 'Query 1' and contains the following SQL code:

```
1 • SELECT Email, 'Customer' AS User_Type
2   FROM Customer
3   UNION
4   SELECT Email, 'Seller' AS User_Type
5   FROM Seller;
6
```

Below the query editor is a 'Result Grid' showing the output of the query. It has two columns: 'Email' and 'User_Type'. The results are as follows:

Email	User_Type
arun.kumar@email.com	Customer
divya.s@email.com	Customer
karthik.m@email.com	Customer
priya.sharma@email.com	Customer
rahul.raj@email.com	Customer
bookstore@email.com	Seller
fashionhub@email.com	Seller
techworld@email.com	Seller

Query 5: UNION ALL

Display all customers and sellers, including all records.

```
SELECT Customer_Name AS Name, Email, 'Customer' AS Type
```

```
FROM Customer
```

```
UNION ALL
```

```
SELECT Seller_Name AS Name, Email, 'Seller' AS Type
```

FROM Seller;

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'SCHEMAS' pane shows a tree view of databases, with 'ecommerce_order_man...' expanded to show tables like 'category', 'customer', 'order_details', 'orders', 'payment', 'product', 'seller', and 'shipping'. The main window shows 'Query 1' with the following SQL code:

```
1 • SELECT Customer_Name AS Name, Email, 'Customer' AS Type
2 FROM Customer
3 UNION ALL
4 SELECT Seller_Name AS Name, Email, 'Seller' AS Type
5 FROM Seller
6
```

Below the query, the 'Result Grid' shows the output of the query. It has columns 'Name', 'Email', and 'Type'. The results are as follows:

Name	Email	Type
Arun Kumar	arun.kumar@email.com	Customer
Priya Sharma	priya.sharma@email.com	Customer
Rahul Raj	rahul.raj@email.com	Customer
Divya S	divya.s@email.com	Customer
Karthik M	karthik.m@email.com	Customer
Tech World	techworld@email.com	Seller
Fashion Hub	fashionhub@email.com	Seller
Book Store	bookstore@email.com	Seller

3. Nested Queries

Query 6: Products Above Average Price

Display products whose price is greater than the average product price.

SELECT

Product_ID,

Product_Name,

Price

FROM Product

WHERE Price > (

SELECT AVG(Price)

FROM Product

);

The screenshot shows a database management interface. On the left, a 'Navigator' pane displays a tree of schemas, with 'ecommerce_order_man...' expanded to show tables like 'category', 'customer', 'order_details', 'orders', 'payment', 'product', 'seller', and 'shipping'. The main area, titled 'Query 1', contains the following SQL query:

```
1 • SELECT
2     Product_ID,
3     Product_Name,
4     Price
5 FROM Product
6 WHERE Price > (
7     SELECT AVG(Price)
8     FROM Product
9 );
```

Below the query editor, a 'Result Grid' displays the results of the query. It has columns for 'Product_ID', 'Product_Name', and 'Price'. The results are as follows:

Product_ID	Product_Name	Price
402	Bluetooth Headphones	2499.00
404	Denim Jeans	1499.00
NULL	NULL	NULL

Query 7: Customers Who Placed Orders

Display customers who have placed at least one order.

SELECT

Customer_ID,

Customer_Name,

Email

FROM Customer

WHERE Customer_ID IN (

SELECT Customer_ID

FROM Orders

);

Query 8: Customers with High-Value Orders

Display customers who have placed orders above the average order amount.

```
SELECT
    Customer_Name,
    Email
FROM Customer
WHERE Customer_ID IN (
    SELECT Customer_ID
    FROM Orders
    WHERE Total_Amount > (
        SELECT AVG(Total_Amount)
        FROM Orders
    )
);
```

4. Advanced Business Reports

Query 9: Total Sales by Product

Generate a report showing the total quantity sold and total revenue for each product.

```
SELECT
    P.Product_ID,
    P.Product_Name,
    SUM(OD.Quantity) AS Total_Quantity_Sold,
    SUM(OD.Subtotal) AS Total_Revenue
FROM Product P
INNER JOIN Order_Details OD
    ON P.Product_ID = OD.Product_ID
GROUP BY
    P.Product_ID,
```

P.Product_Name

ORDER BY Total_Revenue DESC;

The screenshot shows a database management tool interface. On the left, a 'SCHEMAS' pane lists various databases, with 'ecommerce_order_mana' selected. The main area displays a SQL query:

```
1 SELECT
2   P.Product_ID,
3   P.Product_Name,
4   SUM(OD.Quantity) AS Total_Quantity_Sold,
5   SUM(OD.Subtotal) AS Total_Revenue
6 FROM Product P
7 INNER JOIN Order_Details OD
8   ON P.Product_ID = OD.Product_ID
9 GROUP BY
10  P.Product_ID,
11  P.Product_Name
12 ORDER BY Total_Revenue DESC;
13
```

Below the query, a 'Result Grid' shows the following data:

Product_ID	Product_Name	Total_Quantity_Sold	Total_Revenue
402	Bluetooth Headphones	1	2499.00
404	Denim Jeans	1	1499.00
403	Cotton T-Shirt	2	1198.00
406	Java Programming Book	1	950.00
405	Database Management Book	1	850.00
401	Wireless Mouse	1	799.00

Query 10: Customer Purchase Report

Generate a report showing each customer's total number of orders and total amount spent.

SELECT

C.Customer_ID,

C.Customer_Name,

COUNT(O.Order_ID) AS Total_Orders,

SUM(O.Total_Amount) AS Total_Amount_Spent

FROM Customer C

INNER JOIN Orders O

ON C.Customer_ID = O.Customer_ID

GROUP BY

C.Customer_ID,

C.Customer_Name

ORDER BY Total_Amount_Spent DESC;

The screenshot shows a database management tool interface. On the left, the 'SCHEMAS' pane displays a tree view of databases including 'college', 'collegedb', 'company', 'ecommerce_order', and 'ecommerce_order_managem'. The 'ecommerce_order_managem' database is selected, showing tables like 'category', 'customer', 'order_details', 'orders', 'payment', 'product', 'seller', 'shipping', 'Views', 'Stored Procedures', and 'Functions'. The main query editor displays the following SQL query:

```
1 • SELECT
2     C.Customer_ID,
3     C.Customer_Name,
4     COUNT(O.Order_ID) AS Total_Orders,
5     SUM(O.Total_Amount) AS Total_Amount_Spent
6 FROM Customer C
7 INNER JOIN Orders O
8     ON C.Customer_ID = O.Customer_ID
9 GROUP BY
10     C.Customer_ID,
11     C.Customer_Name
12 ORDER BY Total_Amount_Spent DESC;
13
```

Below the query editor, the 'Result Grid' shows the results of the query. The columns are 'Customer_ID', 'Customer_Name', 'Total_Orders', and 'Total_Amount_Spent'. The results are as follows:

Customer_ID	Customer_Name	Total_Orders	Total_Amount_Spent
102	Priya Sharma	1	2499.00
105	Karthik M	1	2449.00
103	Rahul Raj	1	1198.00
104	Divya S	1	850.00
101	Arun Kumar	1	799.00

Query 11: Payment Status Report

Display the total number of payments and total payment amount for each payment status.

SELECT

Payment_Status,

COUNT(Payment_ID) AS Total_Payments,

SUM(Amount) AS Total_Amount

FROM Payment

GROUP BY Payment_Status;

Query 12: Low Stock Product Report

Display products with stock quantity lower than the average stock quantity.

SELECT

Product_ID,

Product_Name,

Stock_Quantity

FROM Product

WHERE Stock_Quantity < (

SELECT AVG(Stock_Quantity)

FROM Product

)

ORDER BY Stock_Quantity ASC;

The screenshot displays a database management interface. On the left, the 'Navigator' pane shows a tree of schemas, with 'ecommerce_order_managem' selected. The main area shows a SQL query in 'Query 1' with line numbers 1 through 11. The query is:
1 SELECT
2 Product_ID,
3 Product_Name,
4 Stock_Quantity
5 FROM Product
6 WHERE Stock_Quantity < (
7 SELECT AVG(Stock_Quantity)
8 FROM Product
9)
10 ORDER BY Stock_Quantity ASC;
11
Below the query, the 'Result Grid' shows the output. It has columns for Product_ID, Product_Name, and Stock_Quantity. The results are:
Product_ID: 406, Product_Name: Java Programming Book, Stock_Quantity: 20
Product_ID: 405, Product_Name: Database Management Book, Stock_Quantity: 25
Product_ID: 402, Product_Name: Bluetooth Headphones, Stock_Quantity: 30
Product_ID: 404, Product_Name: Denim Jeans, Stock_Quantity: 40
A summary row shows NULL for Product_ID, NULL for Product_Name, and NULL for Stock_Quantity. The interface also includes a 'Filter Rows' field and 'Export/Import' buttons.

Product_ID	Product_Name	Stock_Quantity
406	Java Programming Book	20
405	Database Management Book	25
402	Bluetooth Headphones	30
404	Denim Jeans	40
NULL	NULL	NULL

Conclusion

The above advanced SQL queries use:

- **JOINS** to combine data from multiple tables.
- **Set operations** such as UNION and UNION ALL.
- **Nested queries** to perform calculations and comparisons using subqueries.
- **Aggregate functions** to generate business reports such as total sales, customer purchases, payment summaries, and low-stock products.